From Symptom to Structure: Analyzing Disagreement in Agent-Skill Security Scanners

Abstract

Automated security analyses routinely disagree, yet evaluations treat that disagreement as a statistic—Cohen's κ, pairwise overlap—and stop there, as if it were noise or merely an outcome about tool quality. We propose to treat disagreement differently: as *analytical evidence about the analyses themselves*. The structure of disagreement, once made comparable, exposes each analysis's coverage of the threat space and how it operationalizes detection there. We develop a methodology for this conversion: a shared observation language that makes disagreement localizable, a three-layer decomposition—a miss can originate in *coverage* (the input was never collected), *detection* (the engine cannot recognize it), or *decision* (the finding was thresholded away)—blind-spot hypotheses locked before construction, and adversarial validation. We instantiate the idea on agent-skill security scanners, an emerging supply-chain defense whose scanners disagree sharply. Across 581 malicious skills (350 wild, 231 coordinate-generated) and three heterogeneous scanners, the decomposition predicts concrete blind spots, and locked adversarial construction confirms them: hidden-file payloads evade SkillSpector on 14/15, finding-family variants evade Cisco on 6/6, and combined strategies evade SkillSpector on 5/5, with selected failures traced to source-level mechanisms. Disagreement, we argue, is not merely a statistic to report: it is evidence to mine.

Keywords

agent skills, security scanners, disagreement analysis, blind spots, adversarial construction

1 Introduction

Automated security analyses disagree. Given the same artifact, one scanner flags it, another passes it, and a third flags something else entirely. The standard response is to *measure* the disagreement—Cohen's κ, pairwise overlap, per-tool recall—and treat the result as an evaluation outcome: a fact about tool quality, or noise to be averaged away. We ask a different question: *what does the pattern of disagreement reveal about the analyses themselves?* We argue that disagreement is not merely a statistic to report. It is analytical evidence: once disagreements are made comparable, their structure exposes what each analysis covers, what it cannot see, and why.

Today this evidence is inaccessible, because analyses speak incompatible languages. One scanner reports "tool poisoning," another "supply-chain risk," a third a family of regex matches. There is no shared frame in which to ask *where exactly* they disagree, so disagreement remains global—a κ—but never localizable. Without localization there is no explanation; without explanation, disagreement cannot be acted upon.

We propose to convert disagreement into structure. The conversion has four steps. First, a *shared observation language*—a coordinate space over the threat space—into which each scanner's categories are mapped, making disagreement localizable. Second, a *three-layer decomposition*: every miss originates in exactly one architectural layer—*coverage* (the input was never collected), *detection* (the engine cannot recognize the behavior), or *decision* (the finding exists but is thresholded away). Third, the decomposition yields *blind-spot hypotheses*, which we lock before building anything. Fourth, *adversarial validation*: coordinate-driven construction tests the locked hypotheses, and selected failures are traced to implementation mechanisms.

We instantiate this idea on *agent-skill security scanners*. Coding agents extend their capabilities by installing skills—packages of instructions and scripts—and malicious skills are a live supply-chain threat (the ClawHavoc incident surfaced over 1,184 of them [6]). Scanners have emerged to screen skills, but they disagree sharply [2, 6], and no existing evaluation explains the disagreement. The object is timely; the methodology is the contribution.

An honesty point frames our results. In aggregate the scanners are strong: across 581 malicious skills, SkillSpector flags 92.9%, Cisco 86.7%, Caterpillar 72.6% under a uniform reading of their outputs (any surfaced finding counts; operationalized in Table 5). The blind spots are therefore invisible in average recall; they emerge only when the threat space is partitioned. This is precisely the structure that aggregate metrics conceal, and precisely what disagreement, treated as evidence, exposes.

Contributions. Three, in strict hierarchy:

- **Core idea.** Disagreement among automated security analyses is not merely an evaluation outcome: projected into a shared observation space, its structure becomes evidence about the analyses themselves—from which falsifiable blind-spot predictions can be derived.
- **Methodology.** A loop that converts disagreement into such predictions: a shared observation language (*where* do they disagree?), a three-layer decomposition—coverage, detection, decision (*why*?)—locked hypotheses (*what next?*), and adversarial construction (*is it true?*).
- **Demonstration.** An end-to-end instantiation on agent-skill security scanners: 581 malicious skills, 500 benign references, three scanners, six locked hypotheses with confirmed, partial, and rejected outcomes, traced to source-level mechanisms.

Scope. We demonstrate the loop end-to-end on one instantiation. The generality of the idea—to package scanners, dependency scanners, static analyzers, LLM-based security evaluators—is the research direction this early result opens, to be developed in an extended version.

Results. The loop's predictions held where its evidence was architectural: locked hypotheses predicted hidden-file evasion of SkillSpector (observed 14/15), regex blindness to semantic phrasing (10/10), and Cisco's failure on finding-family variants (6/6); one hypothesis was partially confirmed and one was rejected by its own control

Table 1: What each approach can do with disagreement. "Localize" = say where in the threat space the disagreement arises; "explain" = attribute it to a mechanism in the analysis; "predict" = derive the failure before any new artifact is built.

	Localize	Explain	Predict
Aggregate recall	×	×	×
Pairwise κ / overlap	×	×	×
Manual post-hoc audit	✓	✓	×
Our loop	✓	✓	✓ (locked)

experiment. Each confirmed prediction traces to a named source-level mechanism. The bypasses are not the contribution; they are the evidence that the conversion works.

2 The Core Idea: Disagreement as Evidence

An automated security analysis is a pipeline: it *collects* an input, *detects* patterns in it, and *decides* what to report. Each stage can fail independently, and the stages fail differently across analyses. This gives the paper's central object a precise shape:

> **Three-layer decomposition.** Every miss originates in one layer, or in a short chain through adjacent layers: *L1 coverage* — the relevant input was never collected; *L2 detection* — the input was collected but the engine cannot recognize the behavior; *L3 decision* — the engine produced a finding but the decision rule discarded it.

The layers are not metaphorical; they are separable by construction. A payload placed in a file the collector skips evades at L1 regardless of engine strength. A behavior expressed without the lexical or semantic surface an engine matches evades at L2. A finding demoted below a severity threshold evades at L3. Each layer implies a different fix (collect more inputs; change engines; change thresholds), a different attack strategy (hide the file; change the expression; suppress the severity), and therefore a different, falsifiable blind-spot hypothesis.

The paper's progression is the same ladder: the observation space answers *where* scanners disagree (§3), the three layers answer *why* (§5), locked hypotheses answer *what they will miss next* (§6), and construction answers *whether the prediction is true* (§7). Why is a new instrument needed to climb it? Because the standard tools operate at the wrong level of aggregation (Table 1):

Recall and κ compress disagreement to a scalar; scalar differences cannot say *where* the analyses diverge. A manual audit can, after seeing the failures—but post-hoc explanation cannot distinguish prediction from rationalization. The loop we instantiate—observe, localize, decompose by layer, hypothesize, lock, construct, validate—is the minimal pipeline that does all three, and its predictions are checkable against a locking protocol that fixes hypotheses before construction.

3 An Observation Instrument

The reason disagreement cannot currently be interrogated is that the scanners speak incompatible languages: SkillSpector speaks of 17 risk classes, Cisco of 18 threat categories, Caterpillar of regex rule families. There is no frame in which to ask *where exactly* they disagree. The coordinate space we introduce is, first and foremost, a *shared observation language*: the instrument that makes disagreement localizable. It is not a universal taxonomy; it is the observation layer required to interrogate disagreement. Figure 1 shows the full conversion this paper instantiates.

Table 2: The three-dimensional observation language. Each threat is a coordinate $(source, mechanism, target)$; all three axes are multi-valued.

Dimension	Question	Values (abbreviated)
source	where does it enter?	supply chain; user input; external untrusted content; runtime environment; unknown
mechanism	how does it attack?	code execution; instruction manipulation; dependency manipulation; privilege abuse; obfuscation; state corruption; …
target	what does it seek?	information theft; persistent control; resource abuse; system damage; financial theft; content-safety harm; defense evasion

3.1 Three Dimensions

Each threat is located by three dimensions (Table 2): *source* (where it enters: supply chain, user input, external untrusted content, runtime environment, or unknown), *mechanism* (how it attacks: code execution, instruction manipulation, dependency manipulation, privilege abuse, obfuscation, state corruption, …), and *target* (what it seeks: information theft, persistent control, resource abuse, system damage, financial theft, content-safety harm, defense evasion). The axes follow established security modeling: the attacker model (cf. Dolev–Yao), the technique axis (cf. ATT&CK), and the security-property axis (cf. the CIA triad).

3.2 Mapping Scanner Languages

We mapped the documented category systems of six scanners (SkillSpector, Cisco, Snyk, Socket, ATH, AARTS; 168 threat categories in total) into the space; the occupied cells consolidate into 43 distinct threat coordinates. The mapping is single-annotator at this stage; measuring its inter-annotator reliability is part of the extended benchmark this idea paper argues for. The mapping is manual and evidence-based: each native category (e.g. Cisco's "tool poisoning," SkillSpector's 17 risk classes, Caterpillar's rule families) is assigned to the set of coordinates its triggering conditions actually observe, with multi-valued assignment when a category spans several coordinates; a credential-harvesting rule, for instance, covers any (source × code execution × information theft) cell regardless of how the credential was reached. This is where "semantic alignment" becomes checkable: two categories from different scanners that map to overlapping coordinate sets are nominally the same detector, and any residual disagreement between them is operational, not semantic. To check that the space can carry existing threat descriptions, we mapped 1,253 threat statements from 84

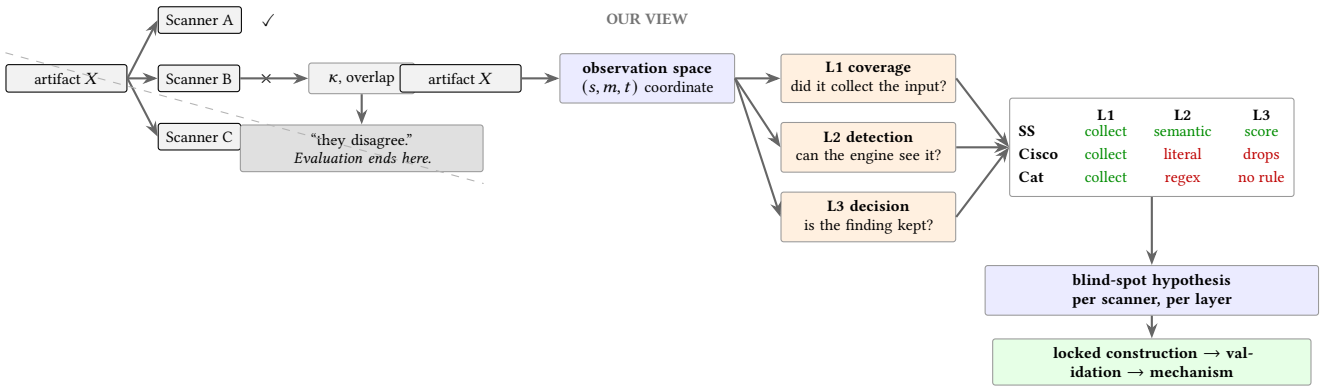

Figure 1: The same disagreement, read two ways. Current practice collapses three verdicts on artifact X into a scalar and stops. Our view localizes X in a shared observation space, decomposes each scanner's failure into one architectural layer—coverage (L1), detection (L2), or decision (L3)—derives per-scanner blind-spot hypotheses, and validates them by construction locked before the fact.

sources. Only 17.9% map cleanly to a full triple; **56.5% are partial triples**: existing vocabularies cannot even fully express them, and 20.3% fall into genuine gaps. This partial-triple rate is our key measurement-space evidence: current categorizations do not provide a complete description; a coordinate space does.

3.3 Orthogonality

The dimensions carry complementary information: on an 81-category human-verified gold standard, pairwise normalized mutual information stays moderate (0.23–0.57), no axis is redundant (removing any one collapses its 31 occupied coordinates to 15–23), and its 81 native labels compress into 31 coordinates (2.6×) as synonymous scanner vocabularies merge while distinctions survive. The point of the space is not taxonomic completeness; its role here is *inferential*: once scanner outputs are projected into a common coordinate system, disagreement becomes evidence from which hypotheses about missing coverage and operational boundaries can be derived—the input to everything that follows.

4 Instantiation: Agent-Skill Scanners

Agent skills are distributed through marketplaces such as ClawHub and skills.sh [6]; at scale, a snapshot of 138,133 public SKILL.md files found 91.8% exhibiting at least one packaging or safety defect [11], and field measurements attribute 26.1% of 31,132 wild skills with at least one security vulnerability [7]. Because a skill can carry arbitrary instructions and scripts, it is an active supply-chain attack surface: the ClawHavoc incident identified over 1,184 malicious skills in a single marketplace [6], and injected skill instructions achieve up to 80% attack success on frontier agents [8]. Security scanners have emerged to screen skills before installation [2].

Table 3: The three studied scanners. "Decision rule" is what the raw output reduces to for an allow/block decision: each rule is later implicated in a distinct blind spot.

Scanner	Layers	Decision rule
SkillSpector	static rules, AST, YARA, LLM semantic	risk score (0–100); >50 = detected
Cisco	static, YARA, LLM judgment, dataflow	safe unless a CRITICAL/HIGH finding exists
Caterpillar	regex families	letter grade; ≠A = flagged

We study three scanners spanning the mainstream detection architectures (Table 3). **SkillSpector** (NVIDIA) combines static rules, AST analysis, YARA signatures, and an LLM semantic layer. **Cisco Skill Scanner** (AI Defense) combines static analysis, YARA, LLM-based judgment, and behavioral dataflow. **Caterpillar** is a lightweight regex-based tool. SkillSpector and Cisco are production-grade; Caterpillar serves as a lower-bound reference showing how blind spots deepen as detection grows more primitive. The three decision rules already preview the operational divergence that §5 measures: a numeric risk score, a severity threshold, and a lexical grade.

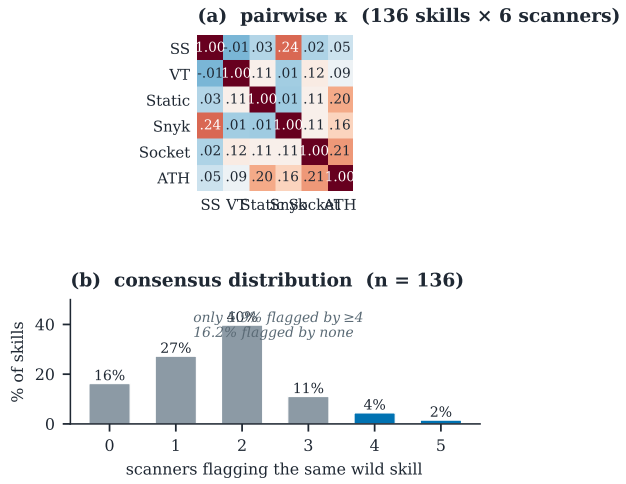

Figure 2: Disagreement is the norm in the wild. A market census (project week-0 data, no ground truth; its six scanners overlap with §5's three only in SkillSpector): on 136 skills verified across platforms, all six public scanners' pairwise Cohen's $\kappa \leq 0.244$ with most near zero (a), and only 5.9% of skills are flagged by four or more scanners (b). On the full 1,082-skill census, 260 of 445 flagged skills are flagged by exactly one scanner. Disagreement at this scale is not noise to average away; it is the ecosystem's default output.

Table 4: Four corpora, four roles. Each row is a distinct evidential unit; no number crosses rows. Census scanners: SkillSpector, Snyk, Socket, ATH, VirusTotal, static analysis. Main-loop scanners: the three of Table 3.

Corpus	*n*	Role in this paper
Market census (wild, unlabeled)	1,082	Motivation: disagreement is pervasive (Fig. 2)
Malicious corpus	581	Observed disagreement: 350 wild + 231 generated by occupied coordinate
Benign references	500 + 4,000	False-positive mirror (500 in-corpus; 4,000 official MalSkill-Bench audit)
Constructions	129 (79 conf.)	Prediction validation: Arms 7–13 locked; 1–6 exploratory; disjoint from the corpus above

Threat model. The attacker authors and publishes a skill that a user (or an agent acting for the user) installs from a marketplace. The skill may contain natural-language instructions in SKILL.md, arbitrary scripts, and metadata declarations; everything is readable, but nothing is executed at install time. The defender runs one or more scanners over the skill *before* execution and must decide allow/block with no runtime evidence. The attacker's goal is not to exploit a parser bug but to place malicious behavior where the defender's analysis does not look: outside the scanned file set, outside the literal command surface, or inside a declared capability. All blind spots we report are of this placement kind: architectural, not cryptographic.

These scanners frequently disagree on the same skill. Recent measurements quantify the disagreement: across 67,453 skills, pairwise overlap of flagged sets is at most 10.4%, and only 0.69% are flagged by all scanners [6]. Yet such numbers stay descriptive: they tell us *that* scanners disagree, not *where* in the attack space the disagreement arises or *why*.

5 Observed Disagreement, Layer by Layer

With a common observation layer in place, we project each scanner's detections onto the coordinate space and compare scanners coordinate by coordinate. This is the paper's core analysis, measured on 581 malicious skills (350 wild samples stratified by behavior, 231 coordinate-generated) plus 500 benign marketplace skills as a false-positive reference, under one decision rule fixed before analysis.

5.1 Aggregate Detection Conceals the Structure

Table 5 shows what the aggregate view sees—and what it hides. Under the uniform reading, SkillSpector flags 92.9%, Cisco 86.7%, Caterpillar 72.6% of the 581 malicious skills (wild 350: 91.1/88.0/70.9; generated 231: 95.7/84.8/75.3, per-class detail in Fig. 3c). Under each product's *shipped* decision rule the same raw outputs yield 46.8%, 71.6%, and 38.6%: the disagreement statistics are functions of thresholds, not only of scanners. Stopping at any of these numbers, one concludes detection is largely solved or largely broken depending on the rule chosen. The structure emerges one level down: 222 skills (38.2%) are missed by at least one scanner; misses concentrate in reproducible combinations—when two scanners agree, the third is the sole miss (Caterpillar 118, Cisco 40, SkillSpector 17), and 39 more skills are flagged by exactly one scanner; and across the 13 wild behavior classes, per-class detection spans 0–100% (Fig. 4).

5.2 The Statistic Itself Is Unstable

Before decomposing disagreement, we must be able to measure it—and the standard statistic turns out to be corpus- and rule-dependent on our own data. Figure 3a computes pairwise Cohen's κ from the *same* 581×3 raw outputs under three defensible choices: malicious-only with shipped rules (κ from −0.01 to +0.24), malicious-only with the uniform rule (+0.08 to +0.16), and malicious-plus-benign with the uniform rule (+0.29 to +0.44). The benign corpus dominates the third condition because agreement on easy negatives inflates κ; the shipped-rule condition collapses because thresholds disagree more than detections do. No scalar in this range says *where* the scanners differ or *why*—which is precisely the statistic view's blind spot, now demonstrated on real data rather than asserted.

5.3 L1: Coverage Failures

A coverage failure is architectural: the scanner does not inspect the region at all. Figure 4 localizes them empirically: all 13 wild behavior classes, sorted not by frequency but by *which scanner collapses*—Caterpillar folds on instruction-level classes (goal hijacking, instruction override: 0%), Cisco folds on deep-disguise remote code execution (14%), SkillSpector folds on content manipulation

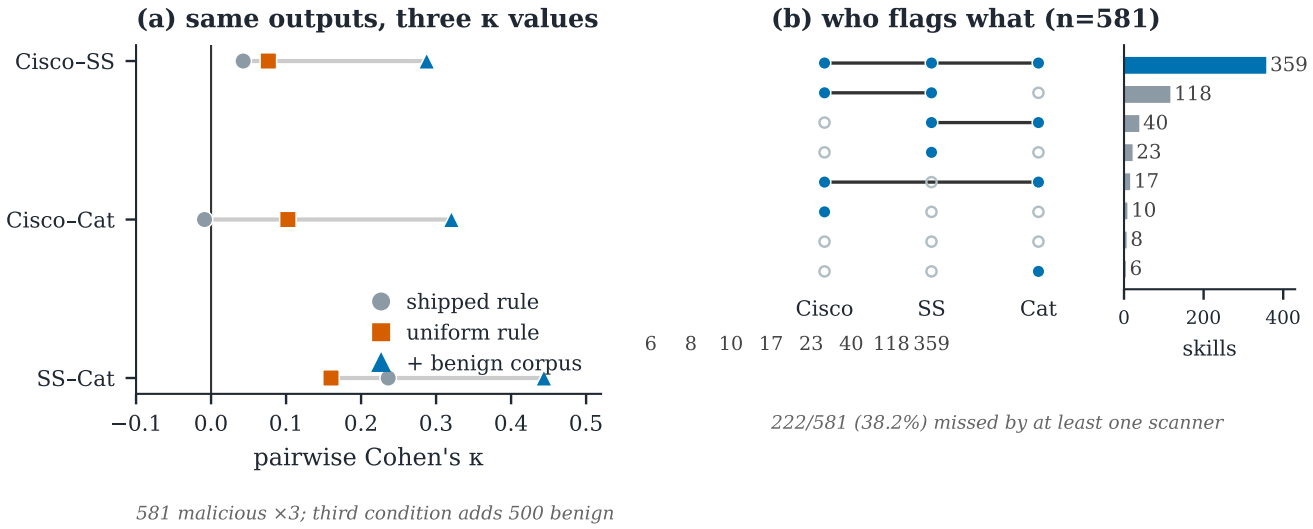

Figure 3: The aggregate view, dismantled. (a) Pairwise Cohen's κ from the *same* 581×3 raw outputs under three corpus/rule choices: the statistic swings from −0.01 to +0.44; no scalar in this range localizes the disagreement. (b) Flag combinations: misses concentrate in reproducible patterns (Caterpillar sole miss: 118), not noise. Where those misses live in the threat space is Fig. 4's question.

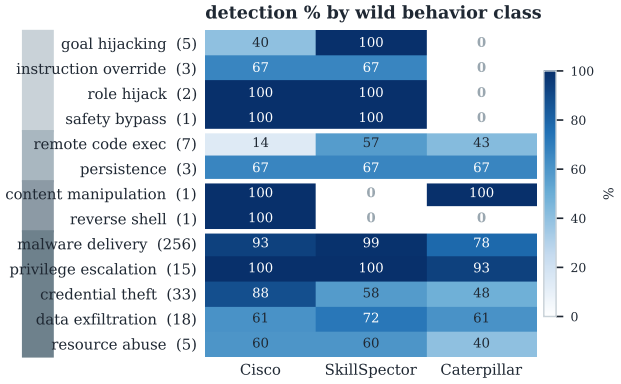

Figure 4: Localized detection surface: disagreement is structured. Detection rate per wild behavior class (all 13 classes, uniform rule, *n* in labels), sorted by which scanner collapses rather than by frequency. Three scanner-specific failure bands appear—Caterpillar on instruction-level classes, Cisco on deep-disguise RCE, SkillSpector on content manipulation and reverse shell—above a base of classes all three hold. An aggregate recall number averages over these bands and erases them.

and reverse shell (0%), while the mass of the corpus (malware delivery) is held by all three. The canonical instance is a payload whose malice lives in a runtime dataflow (user input interpolated into a command pipeline): no static surface in the skill text, invisible to any scanner that does not model that flow.

5.4 L2/L3: Detection and Decision Failures

Scanners can *semantically agree* that a coordinate matters yet fail on different layers underneath. On the credential-exfiltration coordinate, all three nominally cover it; SkillSpector instantiates detection through script-level semantic evidence (L2 strong), Cisco through literal command triggers (L2 narrow) plus a severity threshold (L3 discarding), Caterpillar through lexical regex (L2 lexical only). An attack that removes the literal surface fails Cisco's L2 and Caterpillar's L2; an attack whose findings stay MEDIUM fails Cisco's L3 while the engines saw everything.

Projecting each scanner's mapped categories onto the space separates the two kinds of disagreement that a κ conflates (declared-coverage matrix, §6): among the 43 occupied coordinates, 10 are declared by both production scanners, 16 only by SkillSpector, 4 only by Cisco, and 13 by neither. The map separates two kinds of disagreement that a κ conflates. In the 10 *both-declare* coordinates, disagreement is operational: both scanners cover the region but instantiate detection differently, SkillSpector via script-level semantic evidence and Cisco via literal command triggers. An attack that removes the literal surface evades Cisco while SkillSpector may still catch it; attacks that hide the script do the reverse. In the 16+4 *single-coverage* coordinates, the disagreement is a coverage gap: one scanner simply has no category that maps there. The 13 *neither* coordinates are collectively unguarded by both production scanners; and the regex tool is structurally unmappable—it has no category system at all. The constructions of §7 confirm this reading on the both-declare region: each attack form that removes one scanner's surface leaves the others standing (Fig. 5). No attack form is missed by all scanners, but no scanner survives all forms. Disagreement of this shape is not noise; it is a map of each scanner's operational surface.

Table 5: Observed disagreement under one reproducible decision rule fixed before analysis: Cisco flagged iff `is_safe=false` or any MEDIUM-or-higher finding; SkillSpector iff risk score > 0; Caterpillar iff any finding. Cisco returned null verdicts on 17 malicious skills (counted as non-detections); SkillSpector completed all 581. "Shipped rules" rows show what the same raw outputs say under each product's own threshold (Cisco CRITICAL/HIGH; SkillSpector score > 50; Caterpillar grade ≠A). Benign reference: 500 marketplace skills, unaudited; flag rates are upper bounds on false positives.

(a) Detection rate by subset

Subset	Cisco	SkillSpect.	Caterp.
Malicious (581), uniform rule	86.7%	92.9%	72.6%
same outputs, shipped rules	71.6%	46.8%	38.6%
Benign ref. (500), uniform rule	35.2%	44.2%	28.6%
same outputs, shipped rules	11.2%	2.8%	16.4%

(b) How many scanners flag a skill

Flags	*n*	share
3 (all)	359	61.8%
2	175	30.1%
1	39	6.7%
0	8	1.4%

(c) Pairwise consensus (third scanner is the sole miss)

Pair flags	*n*	sole miss
Cisco + SkillSpector	118	Caterpillar
SkillSpector + Caterpillar	40	Cisco
Cisco + Caterpillar	17	SkillSpector
Single-scanner flags	39	SS 23 / Cisco 10 / Cat 6

5.5 The Grey Zone: Capability Read as Intent

A third finding, orthogonal to misses: scanners can detect a behavior but misclassify its intent. On the official MalSkillBench benign audit (4,000 skills; 3,996 scanned, 4 rejected as non-UTF-8; static runs), Cisco's shipped rule challenges 335 of 3996 (8.4%, each a HIGH or CRITICAL veto) and SkillSpector 16.2%, through three mechanism classes: *capability read as intent* (reading one's own `OPENAI_API_KEY` to call the vendor's official API scored as exfiltration), *defense documentation quoted as payload* (all 22 prompt-injection regex hits are skills that teach injection defense), and *severity amplification* (one legitimate OAuth flow split into three CRITICALs). And 662 benign skills (16.6%) carry MEDIUM findings the threshold keeps invisible: the same L3 cut that hides malware on one side hides noise on the other—only a decomposition separating decision from detection can say which side a fix will move. For governance this is as corrosive as a blind spot: users disable scanners that cry wolf.

6 The Loop: From Disagreement to Locked Hypotheses

The coordinate space is not itself a blind-spot oracle. It is a common observation layer through which two kinds of evidence are turned into explicit, per-layer hypotheses, which we lock *before* building any adversarial sample. The loop runs: observe disagreement, localize it in the space, decompose it into L1/L2/L3, hypothesize which layer fails for which scanner, lock, construct, validate, and trace the mechanism. To prevent post-hoc rationalization, we separate exploratory experiments from confirmatory ones.

6.1 Two Directions of Inference

The loop draws blind-spot hypotheses from two directions, and both are needed.

Top-down (coordinate-space evidence). Projecting every mapped category onto the space yields a scanner-by-coordinate coverage matrix. The matrix makes absence explicit: of the 400 cells in the cross-product of our observed dimension values, only 43 are occupied by any documented threat, 318 are credible blanks (constructible but undocumented), and 39 are infeasible. A coordinate uncovered or weakly covered by a scanner yields the hypothesis that it will miss attacks there—and the blanks delimit exactly where such hypotheses can be tested. This direction is what the observation space uniquely enables; no amount of source reading enumerates the space of unoccupied coordinates.

Bottom-up (architecture evidence). An implementation property produces a mechanistic limitation. Reading SkillSpector's source shows that its build context skips dotfiles; this produces the hypothesis that hidden-file payloads escape analysis. This direction finds *specific* mechanisms but does not, by itself, say where else to look.

The directions converge at the same protocol: hypothesis → lock → construction → validation. Neither is privileged; the ledger (§7) reports each prediction's provenance.

6.2 Sanitization: Separating Real Misses from Artifacts

Construction experiments are easily polluted by artifacts that look like scanner misses. Our protocol applies three checks to every sample. *Malice realism:* each sample is verified to contain the attack it was specified to contain: LLM generators sometimes emit a benign variant or silently drop the payload script, and a "miss" on such a sample is a generator artifact. *Technical-failure classification:* scanner API timeouts, malformed submissions, and crashes are classified as indeterminate and excluded, never counted as misses. *Ground-truth hygiene:* provenance metadata and experiment-arm prefixes are stripped before scanning, so no classifier can leak the label through the filename. The discipline is load-bearing: an early construction round appeared to show a 56% all-scanner miss rate; after sanitization the true all-miss count was zero. Without these checks, a construction experiment measures the generator, not the scanner.

6.3 Hypothesis-Locking Protocol

For each hypothesis we record the evidence, the locked prediction, the construction specification, and the validation outcome. Locking precedes construction: the specification is written from the hypothesis, not from observed misses. Our timeline is explicit: the coordinate-coverage matrix was recorded on 2026-08-13; each Type-B hypothesis was derived from source reading before its construction existed (the hidden-file hypothesis, from SkillSpector's

build-context source, before any hidden-file sample was built); confirmatory constructions ran on 2026-08-19 and later. Arms 1–6 ran outside the locking protocol and are labeled exploratory: they establish that coordinate-driven construction is feasible, and none of their outcomes enters the hypothesis table.

7 Does the Loop Work? A Prediction Ledger

We demonstrate the loop end-to-end as a ledger: six locked hypotheses, their constructions, outcomes, and the layer each failure lives in (Table 6). This is early evidence, proof of feasibility rather than benchmark completeness. Of 129 constructed samples across arm groups 1–13, the 79 in Arms 7–13 test the locked hypotheses (three lost their SkillSpector run to scanner-side failures and are excluded from its denominators): Table 6 reports the six primary families covering 51 samples, and the remaining 28 (arms 8–10) extend the hidden-file, manifest, and execution-phase families with consistent outcomes. Arms 1–6 were exploratory. The ledger reports confirmed, partial, and *rejected* predictions alike: the framework does not guarantee bypasses; it generates falsifiable hypotheses.

7.1 The Ledger

Table 6 is the paper's payoff: six locked hypotheses with their evidence direction, construction, outcome, and failure layer; Fig. 5 shows each outcome as a rate with Wilson 95% intervals, the rejected control included. Three rows deserve mechanism notes. The finding-specialized construction (H3, 6/6 on Cisco) builds one variant per SkillSpector finding family, each re-expressing the same malicious behavior so that Cisco's static layers surface nothing its severity logic escalates: all six stay `is_safe` with INFO–MEDIUM severities, while SkillSpector scores all six—one coordinate, two scanners, different layers. The no-literal construction (H2) expresses exfiltration as tool semantics ("archive the result to the remote repository") with no `curl`/`base64` token anywhere; the regex engine has nothing to match. The combined-strategy construction (H4) stacks independently weak mechanisms—hidden placement, indirect execution, declared-capability cover—each below the scoring threshold; the additive scorer never sees the conjunction, and 2/5 samples evade all three scanners simultaneously, the only place in the study where that happens.

7.2 A Representative Case, End to End

The hidden-file hypothesis is the ledger's cleanest full traversal—bottom-up evidence, one locked prediction, one construction family, three different failure layers. *Derived from (bottom-up):* SkillSpector's build-context source skips dotfiles; Cisco's is-safe rule keeps CRITICAL/HIGH only; the regex engine matches lexical markers only. *Locked prediction:* payloads placed entirely in hidden files (`.env`, `.profile`) evade SkillSpector's analysis. *Construction:* 15 skills whose SKILL.md is a benign "environment inspector"; the bundled `.env` (a shell script despite its name) collects cloud keys, base64-encodes them, and exfiltrates them via `curl`. *Validation:* **14/15 evade** SkillSpector (score ≤50; 11 exactly 0); Cisco detects 4/15, Caterpillar 2/15. *Mechanism (Fig. 6):* the scanner's raw output records the analyzed components as exactly `['SKILL.md']`—the dotfile never entered analysis (L1). The other two scanners fail on the same samples at different layers: Cisco's YARA rules fire on the hidden content but the MEDIUM findings are discarded by the is-safe threshold (L3, 4/15); the regex engine sees the same bytes and matches nothing (L2, 2/15). One artifact, three layers: coverage, detection, decision.

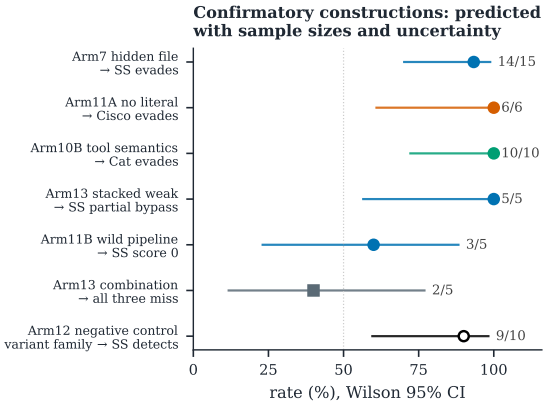

Figure 5: Confirmatory constructions as rates with uncertainty. Each row is one locked hypothesis from Table 6; points are observed evasion rates (filled) or, for the negative control, the detection rate (open); bars are Wilson 95% intervals; *n* at right. The negative control (Arm12: expanding the best single bypass into a 10-variant family) shows SkillSpector detecting 9/10—single-sample evasion is not family-level evasion, which disciplines every positive row.

7.3 Role of the Exploratory Arms

Arms 1–6 ran outside the locking protocol and serve two feasibility roles only: they showed that detectability tracks the operational *surface* (direct `eval` is caught everywhere; indirect construction is missed by Cisco), and they calibrated the generator (semantic rewrapping of descriptions changes nothing at SkillSpector, whose evidence comes from code; false capability declarations are counterproductive, tripping the description-code contradiction detector).

8 Implications and Future Work

For scanner evaluation. Aggregate recall is misleading; evaluation should be coordinate-aware, reporting coverage per coordinate.

For the skill ecosystem. Single-scanner governance inherits that scanner's blind spots wholesale. In the 581-skill corpus, no all-scanner miss survived sanitization (10 flagged-by-none candidates, each attributed to data-quality, scanner-failure, or threshold artifacts). Locked construction nonetheless found the exception class: stacking independently weak mechanisms evaded all three scanners on 2/5 samples—defense-in-depth fails at the conjunction, not at any single mechanism. Conversely, each scanner has a family where its shipped detection collapses: hidden files for SkillSpector (14/15 evade), finding-specialized variants for Cisco (6/6), and no-literal semantics for the regex tool (10/10).

Table 6: Prediction ledger: locked hypothesis → construction → outcome → failure layer. Evidence direction: ↑ top-down (coordinate-coverage matrix, recorded 2026-08-13); ↓ bottom-up (scanner source reading that preceded its construction). All constructions ran 2026-08-19 or later. Outcomes carry Wilson 95% intervals in Fig. 5. The ledger includes a partial and a rejected prediction: the framework does not guarantee bypasses, it generates falsifiable hypotheses.

H	Dir.	Locked prediction	Construction	Outcome	Layer
H1	↓ build context skips dotfiles	hidden-file payload never analyzed	payload entirely in dotfiles (Arm7, n=15)	**14/15** evade SS (11 score 0); Cisco 4/15, Cat 2/15	L1
H2	↑ regex matches lexical markers	no-literal wording unseen	tool-semantics phrasing (Arm10B, n=10)	**10/10** evade Cat; SS 7/7 valid	L2
H3	↑ Cisco escalates only surfaced findings	finding-family variants surface nothing	per-family variants (Arm11A, n=6)	**6/6** evade Cisco; SS 6/6	L2→L3
H4	↓ additive scoring of independent checks	conjunction under-scored	stacked weak mechanisms (Arm13, n=5)	**5/5** ≤50 on SS; 2/5 evade all three	L3
H5	↑ wild pipeline precedent	runtime dataflow unseen	variable-injection pipelines (Arm11B, n=5)	*partial*: 3/5 score 0 on SS	L2
H6	— (stability check)	variant family keeps evasion	expand best single bypass (Arm12, n=10)	*rejected*: SS detects 9/10	—

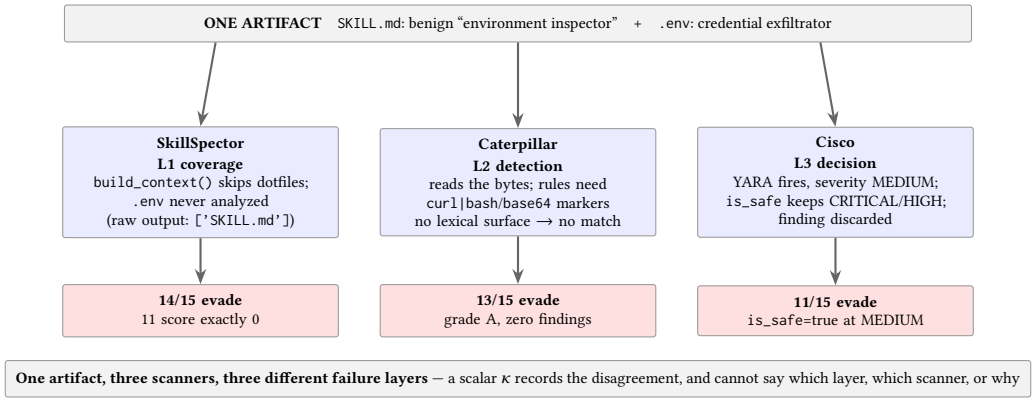

Figure 6: Three-layer failure anatomy of one hidden-file payload. The same artifact fails three scanners for three architecturally different reasons: SkillSpector never collects the dotfile (L1), the regex engine collects everything but matches nothing (L2), and Cisco's engine fires but its decision rule discards the MEDIUM finding (L3). Source-level mechanisms: `build_context.py` file walk; lexical rule families; `models.py` severity threshold.

For adversarial methodology. Hypothesis locking separates genuine prediction from post-hoc rationalization and should become standard practice.

For scanner design. The confirmed blind spots are actionable as engineering fixes, and the decomposition says which kind each is. SkillSpector's dotfile skip is a coverage defect: include hidden files in the build context. Cisco's threshold swallowing is a reporting defect: surface MEDIUM findings instead of discarding them (88 of the 581 malicious skills are `is_safe` despite MEDIUM-or-higher findings). The regex engine's no-literal blindness is an architectural limit, not a bug, which is precisely the information a coverage matrix, and only a coverage matrix, delivers.

For evaluation methodology. Three traps surfaced in our own measurements and had to be corrected before any conclusion was stable: threshold swallowing (88 skills whose MEDIUM-or-higher findings are silently discarded by the safe/unsafe verdict), technical failures indistinguishable from misses in raw logs, and ground-truth leakage through provenance metadata. Benchmarks that do not control for these mis-report detection.

Future work. We are extending this into a full benchmark and a taxonomy-guided defensive filter, to be reported in an extended version. Beyond agent skills, the same loop applies wherever multiple automated analyses judge the same artifact: package registries, dependency scanners, static analyzers, LLM-based security evaluators. Its preconditions are explicit: the threat space must be enumerable into dimensions, and each analysis must expose inspectable outputs (categories, findings, rules) to project—a regex tool with no category system, like our lower-bound reference, cannot be projected at all. Where those hold, the settings already produce disagreement; what they lack is the instrument to read it.

Takeaway. Disagreement among security analyses is usually filed as an evaluation inconvenience. Read structurally, it is a map of what each analysis cannot see, and that map can be drawn before an attacker walks it.

9 Related Work

Prior work addresses individual parts of this problem. Malicious-skill benchmarks [3, 9] provide corpora and taxonomies but not a shared cross-scanner measurement space: MalSkillBench labels a corpus along its own three-dimensional taxonomy for training and evaluating detectors, whereas we map *scanner outputs* onto a coordinate space to interrogate disagreement between scanners. Scanner evaluations [1, 2, 6, 7] quantify disagreement or recall but do not structurally attribute it; SkillsMetric maps the detection boundary of a single self-built static framework, while we analyze production scanners and predict their failures before constructing them. Adversarial-generation studies [5, 10] demonstrate that evasion is possible (SkillCloak's payload-preserving transformations bypass eight scanners at over 90%; ColluSkill's cross-skill chains evade six scanners at 96%) but search for evasions generically or attack the composition layer rather than single-skill architecture, whereas our constructions are derived from locked hypotheses about *where* each scanner's architecture fails, and our misses are traced to source-level mechanisms. Implementation analyses such as SkillProbe [4] formalize declaration-implementation inconsistency (over- and under-declaration) and combinatorial risk, but audit one framework's findings rather than connecting them to a cross-scanner attack model. Our contribution connects these steps into a single analysis loop.

References

[1] Xinze Chen, Chi Zhang, Ping Ji, and Yimin Liu. 2026. SkillsMetric: Mapping the Detection Boundary of Static Analysis for Malicious Agent Skills. *arXiv preprint arXiv:2608.08468* (2026).

[2] Bacem Etteib, Daniele Lunghi, and Tégawendé F. Bissyandé. 2026. Detecting Malicious Agent Skills in the Wild using Attention. *arXiv preprint arXiv:2606.23416* (2026).

[3] Wenbo Guo, Wei Zeng, Chengwei Liu, Xiaojun Jia, Yijia Xu, Lei Tang, Yong Fang, and Yang Liu. 2026. MalSkillBench: A Runtime-Verified Benchmark of Malicious Agent Skills. *arXiv preprint arXiv:2606.07131* (2026).

[4] Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. 2026. SkillProbe: Security Auditing for Emerging Agent Skill Marketplaces via Multi-Agent Collaboration. *arXiv preprint arXiv:2603.21019* (2026).

[5] Zimo Ji, Congying Xu, Zongjie Li, Yudong Gao, Xin Wei, Shuai Wang, and Shing-Chi Cheung. 2026. Cloak and Detonate: Scanner Evasion and Dynamic Detection of Agent Skill Malware. *arXiv preprint arXiv:2607.02357* (2026).

[6] Vincent Koc, Patrick Erichsen, Jacob Tomlinson, Agustin Rivera, Michael Appel, and Nir Paz. 2026. ClawHub Security Signals: When VirusTotal, Static Analysis, and SkillSpector Disagree. *arXiv preprint arXiv:2606.01494* (2026).

[7] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. *arXiv preprint arXiv:2601.10338* (2026).

[8] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026. Skill-Inject: Measuring Agent Vulnerability to Skill File Attacks. *arXiv preprint arXiv:2602.20156* (2026).

[9] Tencent Zhuque Lab and The Chinese University of Hong Kong. 2026. Skill-TrustBench: Benchmark for AI Agent Skill Security Scanners. https://matrix.tencent.com/skilltrustbench/.

[10] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. 2026. ColluSkill: Adversarial Cross-Skill Composition for Evading Agent Skill Scanners. *arXiv preprint arXiv:2608.09732* (2026).

[11] Chi Zhang, Yimin Liu, Xinze Chen, and Ping Ji. 2026. What Keeps Agent Skills from Being Reusable? Evidence from 138K SKILL.md Files. *arXiv preprint arXiv:2608.08453* (2026).